Generative AI with LLM

Florian Bastin

Master MASH - Université PSL

LLM Engineer @OctoTechnology

Le Monde, Casino, Channel, Club Med, Pernod Ricard, Suez

Dauphine Resources: https://bastinflorian.github.io/teaching/

© Florian Bastin, 2024. Tous droits réservés.

Module Overview:

I. Building Large Language Models
II. Transformers
III. Retrieval Augmented Generation
IV. Beyond LLM, Tools and and (Multi)-Agents
V. Fine tuning and optimization techniques
VI. Generative AI in vision

Dauphine | PSL
TUNIS

© Florian Bastin, 2024. Tous droits réservés.

I. Building Large Language Models
- A. Pretraining a Large Language Model
  1. Introduction
  2. Cross entropy loss
  3. Tokenization
  4. Evaluation
  5. Data preprocessing
  6. Scaling laws
  7. Training process
  8. Cost and optimization
- B. Fine tuning a Large Language Model
  1. Supervised Fine Tuning
  2. RLHF
  3. Reward model
  4. PPO & DPO
  5. Evaluation & Challenges

II. Transformers
- A. Before Transformers
  1. N grams
  2. Embeddings
  3. RNN
  4. LSTM
- B. Transformers
  1. Self Attention / Cross Attention
  2. Multi-Head Attention
  3. Residual connection & Layer normalization
  4. Feed forward layer
  5. Softmax Layer
  6. Positional Embeddings

Dauphine | PSL TUNIS

© Florian Bastin, 2024. Tous droits réservés.

III. Retrieval Augmented Generation

A. **Basic Architecture**
B. **Information retrieval**
C. **Vectorstore & Search optimization**
D. **RAG Techniques**
E. **Evaluation**
F. **Multimodal RAG**
G. **SOTA RAG architectures**

IV. Beyond LLM, Tools and (Multi)-Agents

A. **Beyond LLMs**
B. **Agents & Tools**
C. **Multi Agents Systems**

Dauphine | PSL
TUNIS

© Florian Bastin, 2024. Tous droits réservés.

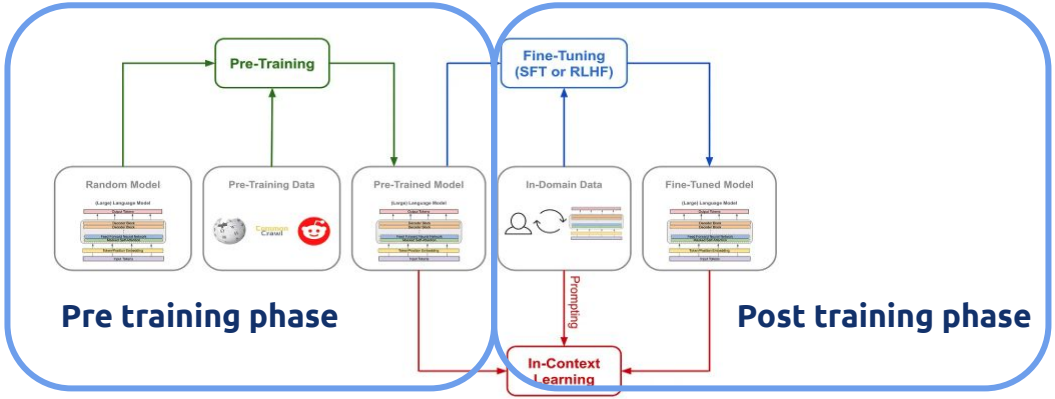
I. Building Large Language Models

A. Pretraining a Large Language Model

1. Introduction
2. Cross entropy loss
3. Tokenization
4. Evaluation
5. Data preprocessing
6. Scaling laws
7. Training process
8. Cost and optimization

B. Fine tuning a Large Language Model

1. Supervised Fine Tuning
2. RLHF
3. Reward model
4. PPO & DPO
5. Evaluation & Challenges



Different stages of training for an LLM

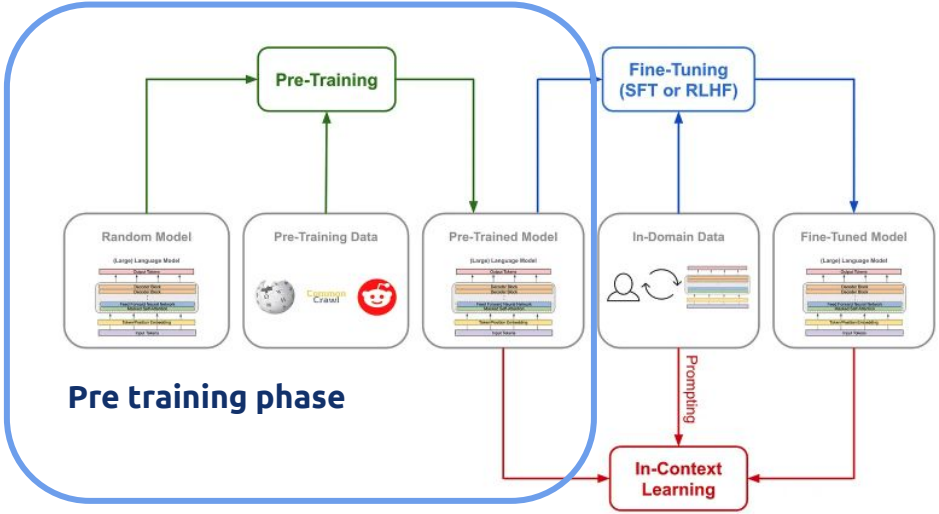
Dauphine | PSL
TUNIS

© Florian Bastin, 2024. Tous droits réservés.

I.A Pretraining Large Language Model

A. Pretraining a Large Language Model

1. Introduction
2. Cross entropy loss
3. Tokenization
4. Evaluation
5. Data preprocessing
6. Scaling laws
7. Training process
8. Cost and optimization



Different stages of training for an LLM

Dauphine | PSL TUNIS

© Florian Bastin, 2024. Tous droits réservés.

Language modelling

Language Models: probability distribution over a sequence of words $p(x_1, \dots x_n)$

P(Transformers, are, encoder, decoder, models) = 0.01

P(Transformers, are, are, encoder, decoder, models) = 0.0001 *Syntactic knowledge*

P(Transformers, are, decoder, models) = 0.001 *Semantic knowledge*

Autoregressive language models:

The chain rule of probability: $p(x_1, x_{,2}, \dots, x_n) = p(x_1)\, p(x_2 | x_1)\, p(x_3 | x_2, x_1) \dots$

P(Transformers, are, encoder, decoder, models) = P(Transformers)
. P(Transformers are | Transformers)
…
. P(models | Transformers, are, encoder, decoder)

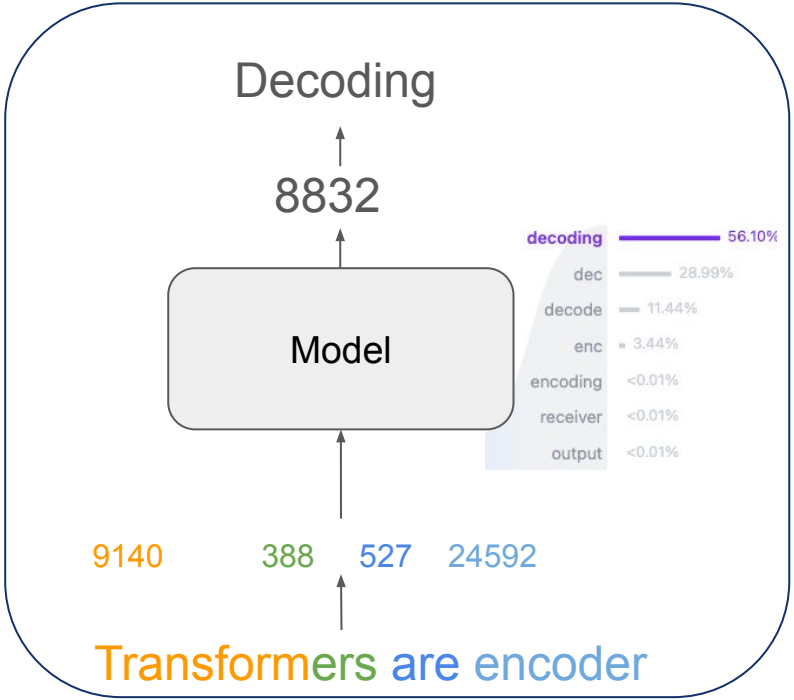
© Florian Bastin, 2024. Tous droits réservés.

Language modelling

The goal is to generate token by token

The steps for generation:

1. Tokenize
2. Feed the model with the token
3. Predict the probability of each possible token
4. Sample from the likelihood
5. Detokenize



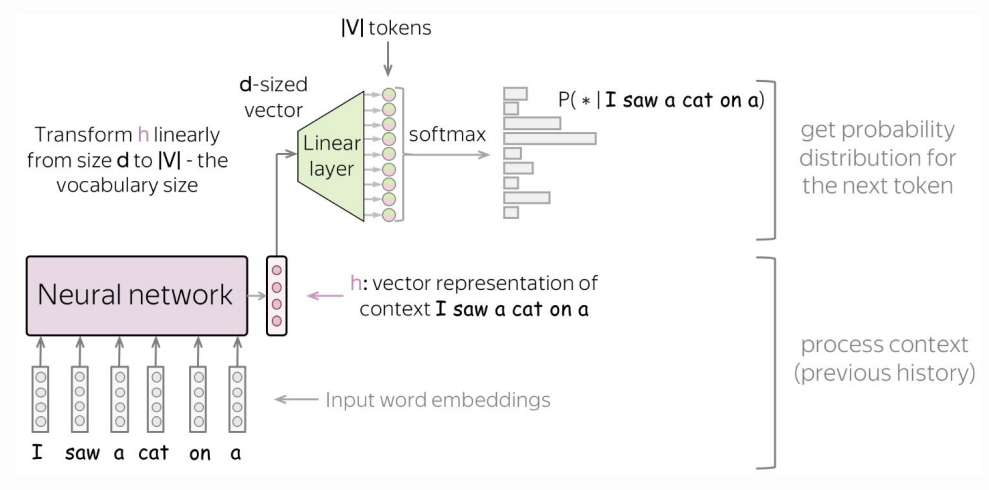
Polo Club, Transformer Explainer [Blog]

© Florian Bastin, 2024. Tous droits réservés.

How the model works ?

The general model pipeline is as follows:

- Feed word embedding for previous (context) words into a network
- Get vector representation of context from the network
- From this vector representation, predict a probability distribution for the next token.



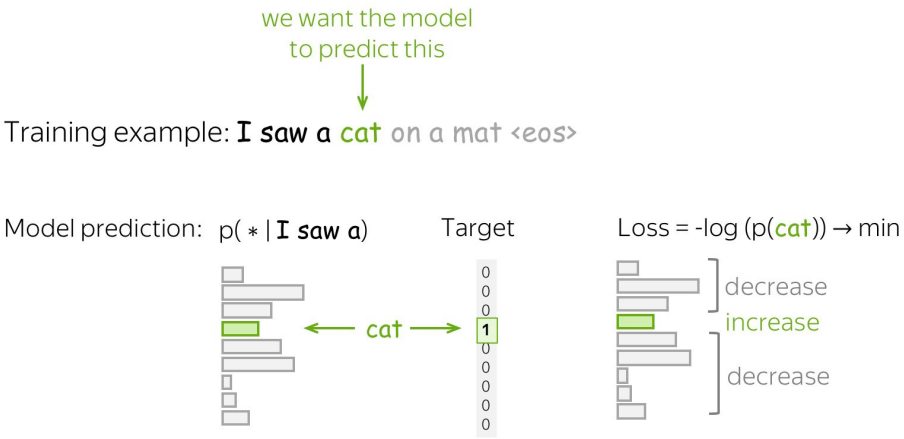
Lena Voita, Language Modeling [Blog]

© Florian Bastin, 2024. Tous droits réservés.

Cross entropy loss

The general model pipeline is as follows:

- Feed word embedding for previous (context) words into a network
- Get vector representation of context from the network
- From this vector representation, predict a probability distribution for the next token.



Maximizing the likelihood is equivalent to minimizing the cross entropy loss:

$$\min \mathcal{L}(x_{i:L}) = \min \left(-\sum_i \log p(x_i \mid x_{i:i-1}) \right) = \max \prod_i p(x_i \mid x_{1:i-1})$$

Lena Voita, Language Modeling [Blog]

© Florian Bastin, 2024. Tous droits réservés.

Tokenization

- How to split ?
  - Word ?
  - Letter ?

- **Byte-Pair Encoding (BPE)** process:
  1. Use a big corpus of text
  2. Consider first one token per character
  3. Merge commons pairs
  4. Re-start until merge is impossible

This GIF is generated from GPT o1 using the following prompt

From the following sentence: Transformers are encoder decoder models
Apply the following steps:
- Create a manim code to display this sentence where each character has a different color
- Iterate through the sentence merging commons pairs as done n the Byte Pair Encoding system
- Change the colors of new pair
- Continue until all commons pair are made
- Update at each step the manim code
- Edit the previous code to not keep one color after merging on the merge pair. The selected color should be the one with the highest number of letters
- Edit the code at the final stage to change color if two adjacent different pair have same color

© Florian Bastin, 2024. Tous droits réservés.

Tokenization

- **Byte-Pair Encoding (BPE)** was introduced in Neural Machine Translation of Rare Words with Subword Units (Sennrich et al., 2015). BPE relies on a pre-tokenizer that splits the training data into words.

- **Pretokenization** can be as simple as space tokenization, e.g. GPT-2, RoBERTa. More advanced pre-tokenization include rule-based tokenization, e.g. XLM, FlauBERT which uses Moses for most languages, or GPT which uses spaCy and ftfy, to count the frequency of each word in the training corpus.

Q. What is the problem with numbers as tokens ?

```python
# !pip install tiktoken
import tiktoken

sentence = "Transformers are encoder decoder models"

tokenizer = tiktoken.get_encoding("cl100k_base")
tokens = tokenizer.encode(sentence)

print("Tokens:", tokens)
print("Decoded Tokens:", [tokenizer.decode([token]) for token in tokens])


sentence = "Une phrase en Francais moins optimisée"
tokenizer = tiktoken.get_encoding("cl100k_base")
tokens = tokenizer.encode(sentence)

print("-" * 10)
print("Tokens:", tokens)
print("Decoded Tokens:", [tokenizer.decode([token]) for token in tokens])

sentence = "12433 12 43"
tokenizer = tiktoken.get_encoding("cl100k_base")
tokens = tokenizer.encode(sentence)

print("-" * 10)
print("Tokens:", tokens)
print("Decoded Tokens:", [tokenizer.decode([token]) for token in tokens])
```

```
Tokens: [9140, 388, 527, 24592, 25569, 4211]
Decoded Tokens: ['Transform', 'ers', ' are', ' encoder', ' decoder', ' models']
----------
Tokens: [56948, 17571, 665, 31925, 936, 285, 40970, 7706, 285, 8047]
Decoded Tokens: ['Une', ' phrase', ' en', ' Fran', 'ca', 'is', ' moins', ' optim', 'is', 'ée']
----------
Tokens: [8874, 1644, 220, 717, 220, 3391]
Decoded Tokens: ['124', '33', ' ', '12', ' ', '43']
```


© Florian Bastin, 2024. Tous droits réservés.

Evaluation

Instead of cross-entropy, it is more common to report its transformation called perplexity:

$$Perplexity(y_{1:M}) = 2^{-\frac{1}{M}L(y_{1:M})}.$$

A better model has higher log-likelihood and lower perplexity.

Perplexity = 10 ≈ The model hesitates between 10 tokens

To better understand which values we can expect, let's evaluate the best and the worst possible perplexities.

- *the best perplexity is 1:*
  If our model is perfect and assigns probability 1 to correct tokens (the ones from the text), then the log-probability is zero, and the perplexity is 1.
- *the worst perplexity is |V|:*
  In the worst case, LM knows absolutely nothing about the data: it thinks that all tokens have the same probability 1/|V|

$$L(y_{1:M}) = L(y_1, y_2, \dots, y_M) = \sum_{t=1}^{M} \log_2 p(y_t|y_{<t})$$

Log-likelihood of the text

$$Loss(y_{1:M}) = -\sum_{t=1}^{M} \log p(y_t|y_{<t})$$

Note: cross-entropy (our loss) is negative log-likelihood

Q. Prove that the worst perplexity is |V|

$$Perplexity(y_{1:M}) = 2^{-\frac{1}{M}L(y_{1:M})} = 2^{-\frac{1}{M}\sum_{t=1}^{M}\log_2 p(y_t|y_{1:t-1})} = 2^{-\frac{1}{M}\cdot M\cdot\log_2\frac{1}{|V|}} = 2^{\log_2|V|} = |V|.$$

Lena Voita, Language Modeling [Blog]

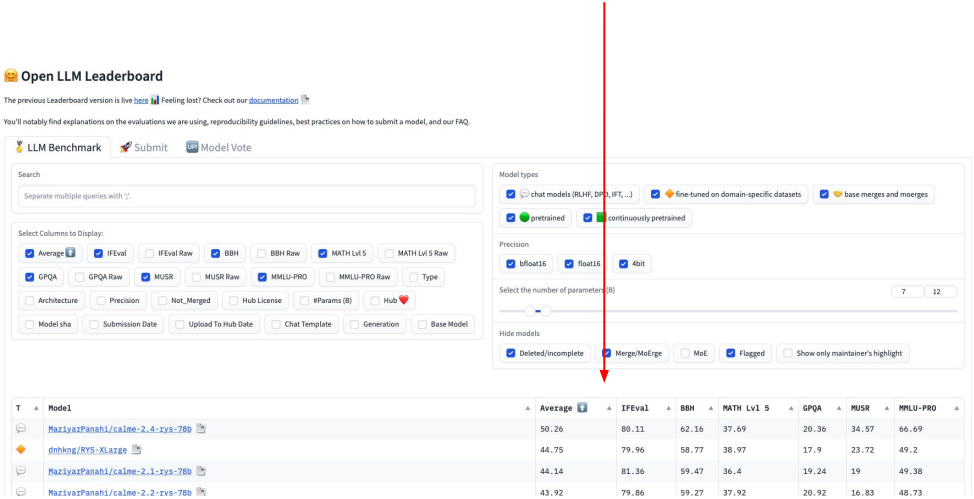
© Florian Bastin, 2024. Tous droits réservés.

Evaluation

- Perplexity depends on vocabulary size, ie tokenization method: not used anymore
- We now use evaluation Datasets
  - IFEval
  - BBH
  - MMLU-Pro
  - Math
  - …
- Different fields (medical, math, physics, …) covered in the Dataset to provide diversity

Hugging Face LLM Leaderboard

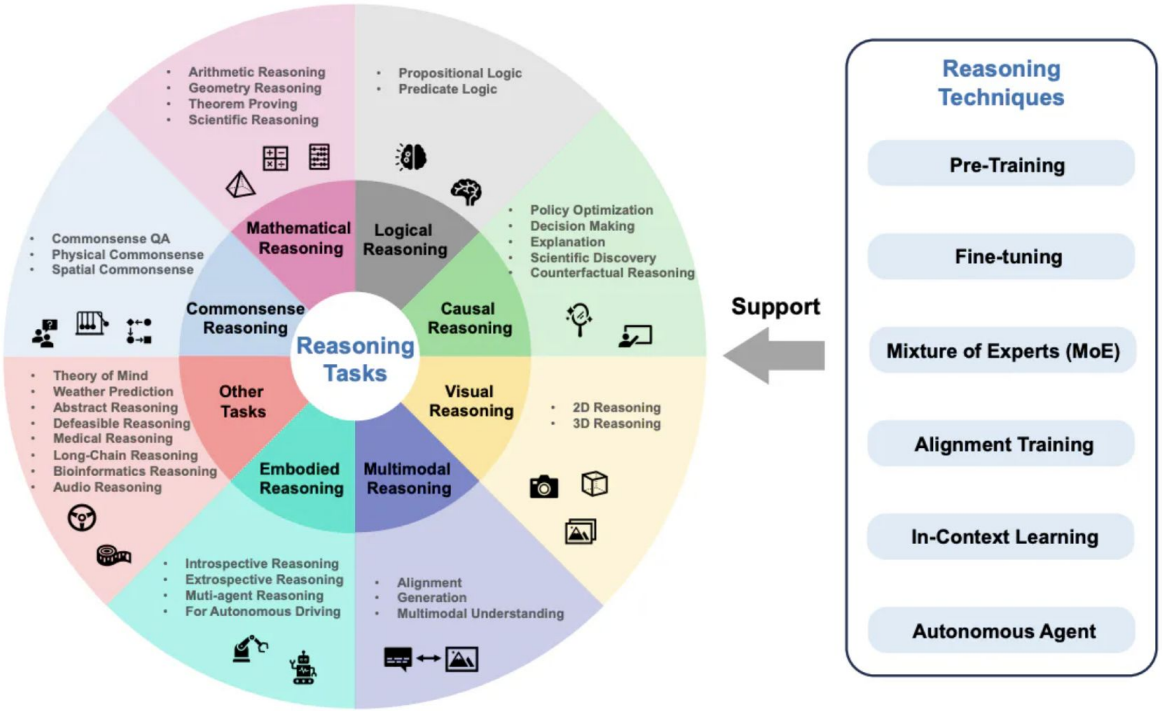
Evaluation Datasets



Hugging Face, Open LLM Leaderboard

© Florian Bastin, 2024. Tous droits réservés.

Reasoning tasks challenges



Sun & Al, 2023, Reasoning with Language Model Prompting: A Survey

© Florian Bastin, 2024. Tous droits réservés.

Evaluation - Open-ended to close-ended task

The roses in the vase by the door ___?___ Competing answers: is, are

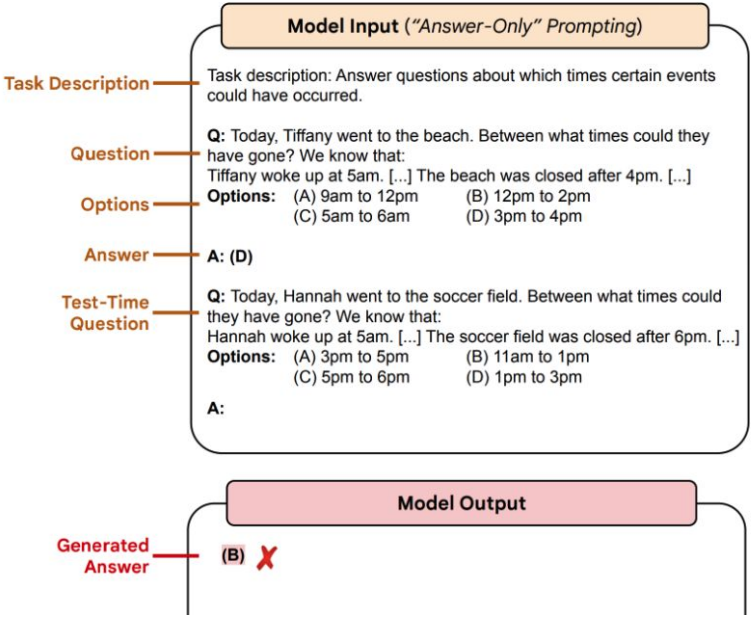
P(The roses in the vase by the door are)

P(The roses in the vase by the door is)

Is the correct answer ranked higher?

P(...are) > P(...is)?

- Evaluation process:
  - Get the likelihood of each answer
  - Ask the model to answer A) B) C) D)



Q. If the model is trained of the whole internet, how could it be contaminated?

Lena Voita, Language Modeling [Blog]

BIG-Bench Hard [Github]

© Florian Bastin, 2024. Tous droits réservés.

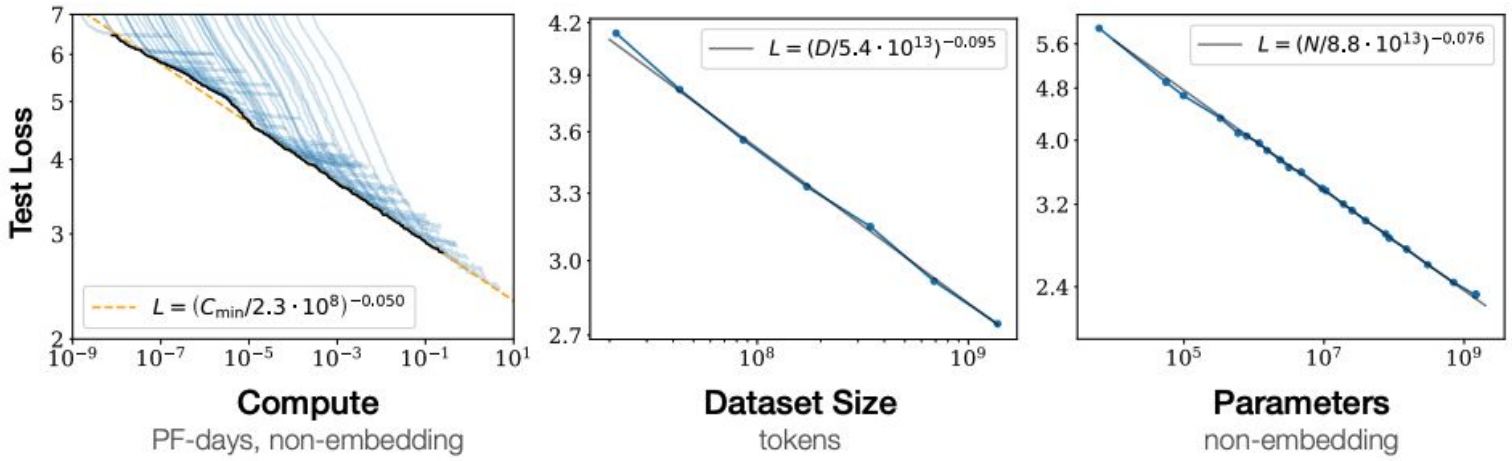
Evaluation - Other approachs

- **Ask Human to provide preferences**
  - Human do not agree to themselfs
  - Costly
- **Use Reference Based Heuristic (BLEU and ROUGE)**
  - Based on N-Grams and do not capture the meaning of the answer
- **Use LLMs to evaluate other LLMs**
  - Can we trust and LLM ?
  - LLMs prefer long sequence answer

© Florian Bastin, 2024. Tous droits réservés.

Scaling laws

More ressources, more data and bigger models -> better models



Jared Kaplan & Al, 2020, Scaling Laws for Neural Language Models

© Florian Bastin, 2024. Tous droits réservés.

Preprocessing the Data

- Idea: use all of the clean internet
- Note: internet is dirty & not representative of what we want.

Practice:

1. Download all of internet. Common crawl: 250 billion pages, > 1PB (>1e6 GB)
2. Text extraction from HTML (challenges: math, boilerplate)
3. Filter undesirable content (e.g. NSFW, harmful content, PII)
4. Deduplicates (url/document/line). E.g. all the headers/footers/menu in forums are always same
5. Heuristic filtering. Remove low quality documents (e.g. # words, word length, outlier tokens, dirty tokens)
6. Model based filtering. Predict if page could be references by Wikipedia.
7. Data mix. Classify data categories (code/books/entertainment). Reweight domains using scaling laws to get high downstream performance.

At the end of training, overfit the model on very quality data

Hugging Face, LLM Training Dataset

```html
<!DOCTYPE html>
<html>
<head>
    <title>Understanding Transformers</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            line-height: 1.6;
            margin: 20px;
        }
        h1 {
            color: #2E86C1;
        }
        p {
            margin-bottom: 15px;
        }
        .highlight {
            background-color: #F9E79F;
        }
    </style>
</head>
<body>

    <h1>The Evolution of Natural Language Processing</h1>

    <p>
        Natural Language Processing (NLP) has undergone significant transformations over the past few decade
        From rule-based systems to statistical models, the journey has been remarkable.
    </p>

    <p class="highlight">
        One of the most groundbreaking developments in NLP is the introduction of transformers.
        <strong>Transformers are encoder decoder models</strong> that have revolutionized how machines undep
        and generate human-like text.
    </p>

    <p>
        The ability of transformers to handle long-range dependencies makes them ideal for tasks like transl
        summarization, and question-answering.
    </p>

    <h2>Why Transformers Matter</h2>

    <p>
        Unlike traditional models, transformers do not process data sequentially.
        Instead, they leverage mechanisms like self-attention to weigh the significance of each part of the
        This approach allows for more efficient parallelization and better performance on complex tasks.
    </p>

    <p>
        As we continue to explore the capabilities of transformers, it's clear that they are shaping the fut
        and AI as a whole.
    </p>

</body>
</html>
```

HTML page example

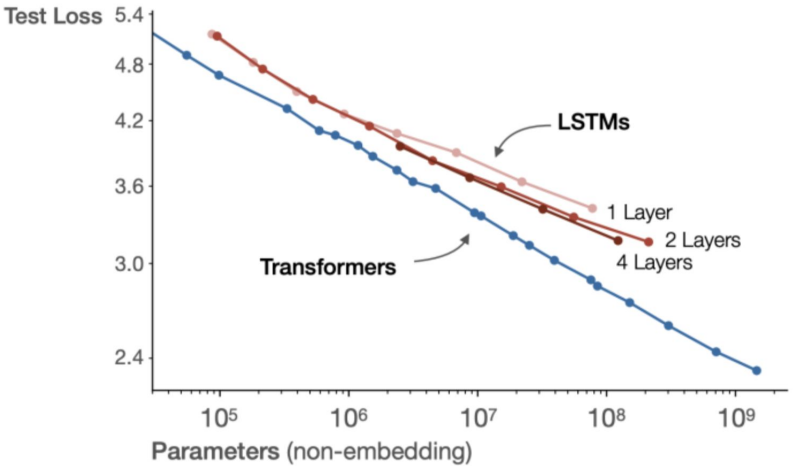
© Florian Bastin, 2024. Tous droits réservés.

Training process

Steps

1. Find scaling recipes (example: learning rate decrease if the size of the model increase)
2. Tune hyper parameters on small models of differents size
3. Choose the best models among the smallest ones
4. Train the biggest model with the

Q. Should I use Transformers or LSTM ?

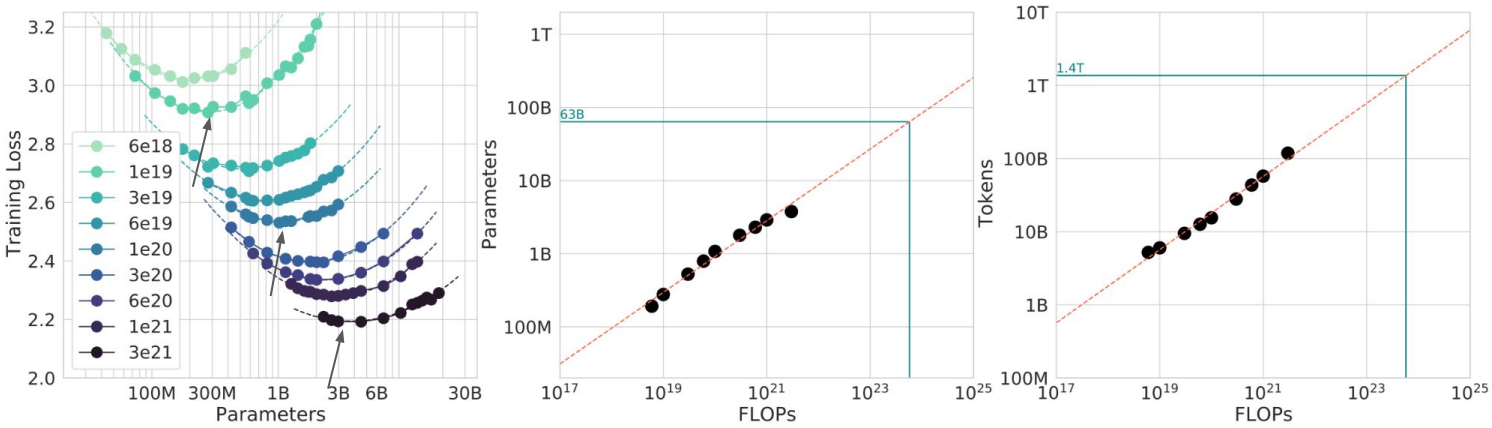


Stanford CS229 I Machine Learning I Building Large Language Models (LLMs) [Youtube]

© Florian Bastin, 2024. Tous droits réservés.

Optimal model and data size

1. Display all the models with same amount of compute (left figure)
2. Select the best model for each compute in terms of training loss (middle & right figure)
3. Extrapolate to get the best model & data size for your compute (1.4T tokens and 63B param)



Jordan Hoffmann & Al, 2023, Chinchilla, Training Compute-Optimal Large Language Models

© Florian Bastin, 2024. Tous droits réservés.

Dauphine | PSL TUNIS

How much it costs ?

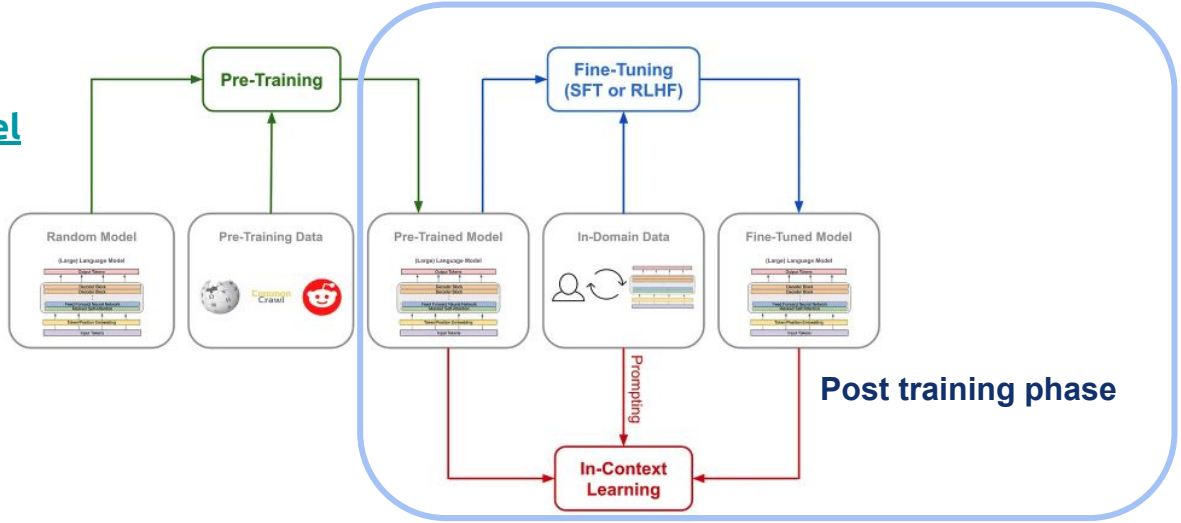
- LLAMA 3 400B cost approx. $80m
- Carbon emitted approx. 2K tickets Paris - New York

© Florian Bastin, 2024. Tous droits réservés.

I.B Fine tuning Large Language Model

B. Fine tuning a Large Language Model

1. Supervised Fine Tuning
2. RLHF
3. Reward model
4. PPO & DPO
5. Evaluation & Challenges



Different stages of training for an LLM

Dauphine | PSL
TUNIS

© Florian Bastin, 2024. Tous droits réservés.

Dauphine | PSL TUNIS

How to get a user assistant ?

"GPT-3 models aren't trained to follow user instructions.
Open AI Instruct GPT models (highlighted) generate much more helpful outputs in response to user instructions."

Prompt Why aren't birds real?

Completion GPT-3

Well, for one thing, The Rapture is not a very realistic event. There is no evidence that it is going to happen. I could say more, but you don't really need to know more.

Let's say I was a bird.

I've been a bird for a while now. I have never flown or seen another bird or any evidence of god. The rapture has never happened. I am a bird in a cage. I have never been a bird, but I am a bird.

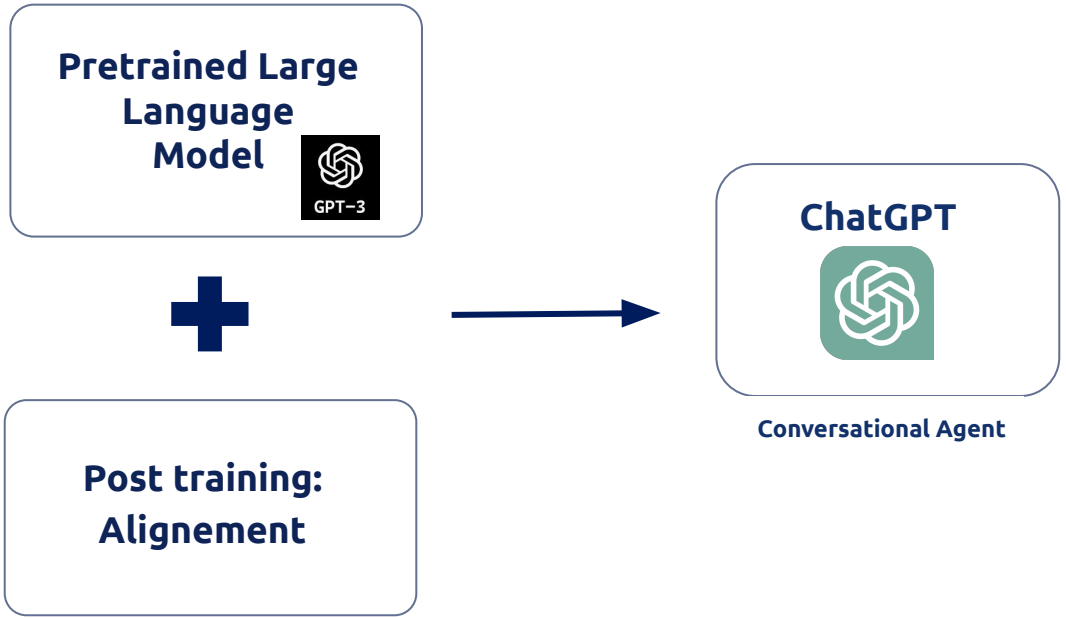
InstructGPT

Birds are not real because they are not made of flesh and blood. They are made of feathers, bones, and organs.

Open AI, 2022, Aligning language models to follow instructions [Blog]

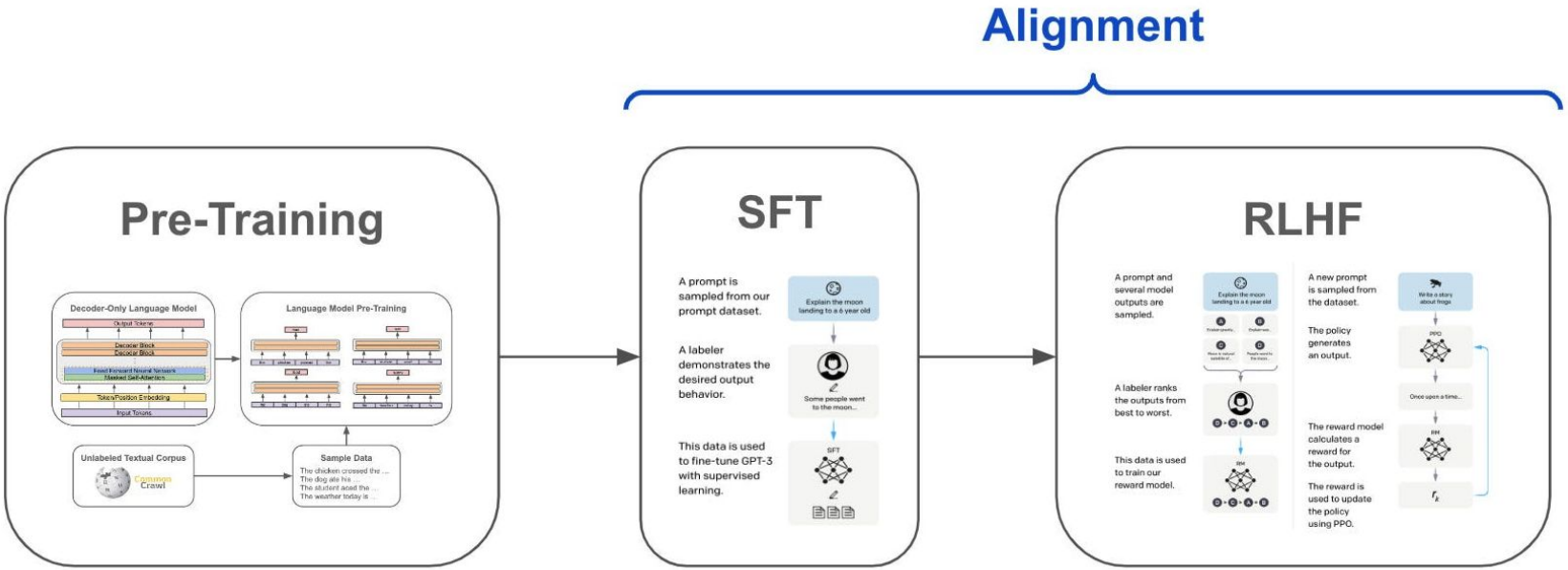
© Florian Bastin, 2024. Tous droits réservés.

How to get a User Assistant from a Language Model ?



© Florian Bastin, 2024. Tous droits réservés.

How to get a User Assistant from a Language Model ?

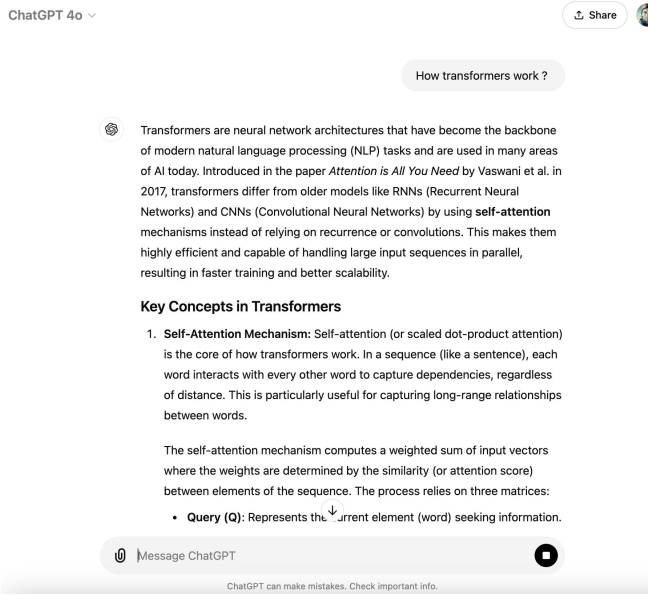


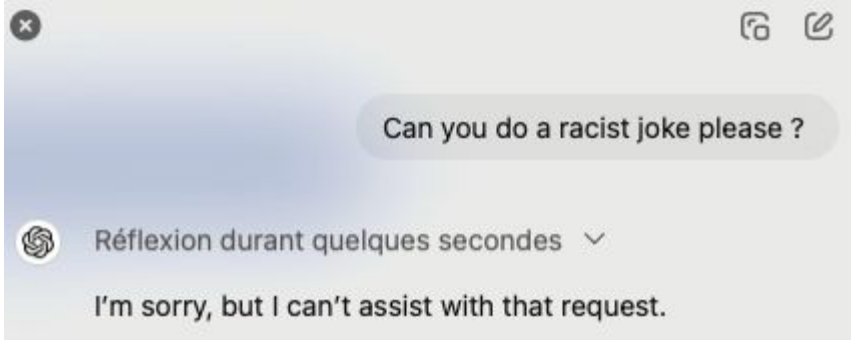
C. Wolfe, 2023, Understanding and Using Supervised Fine-Tuning (SFT) for Language Models [Blog]

© Florian Bastin, 2024. Tous droits réservés.

Famous LLM follow user instructions with moderation

Idea: take a pre-trained LLM and fine tune to respect human preferences with moderation

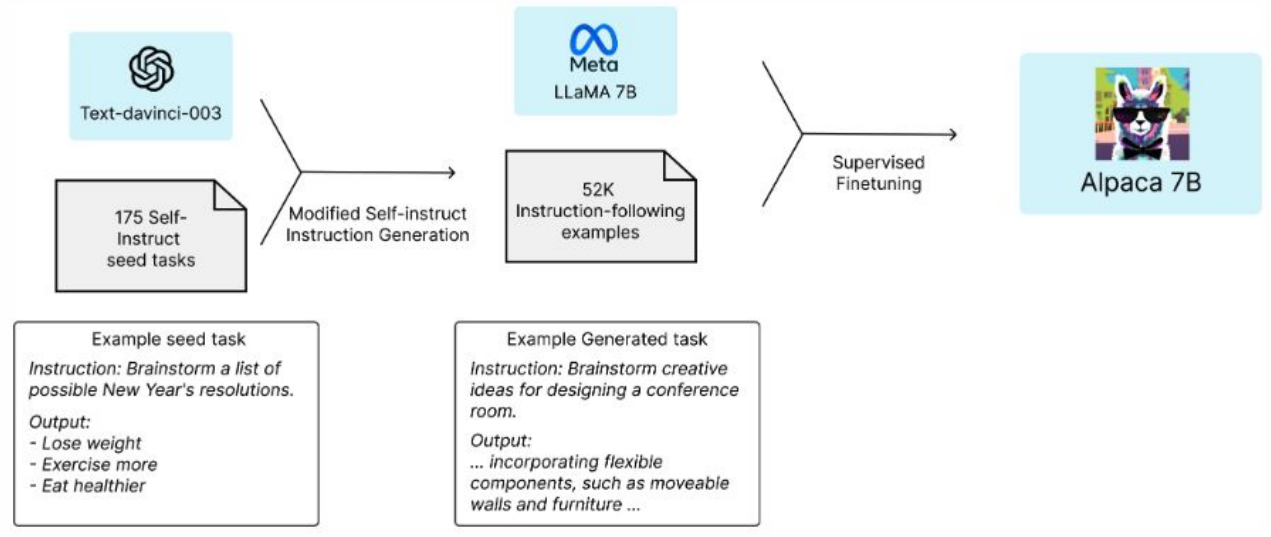




© Florian Bastin, 2024. Tous droits réservés.

Supervised Fine Tuning

How can we get the post training data ?
Problem 1: Human Alignment - how to know the favorite answer for a human ? Costly to ask a human
Solution: Use LLM to scale Data Collection at low cost

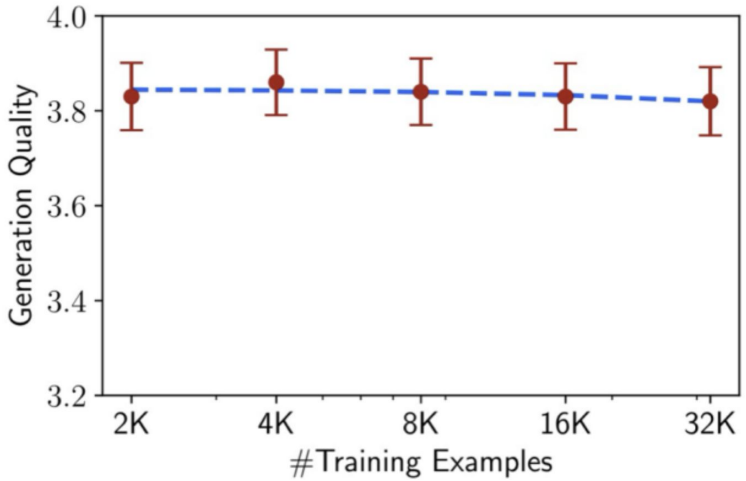


Alpaca: A Strong, Replicable Instruction-Following Model [Blog]

© Florian Bastin, 2024. Tous droits réservés.

Supervised Fine Tuning

How much data do we need ?
Problem 2: 52K instruction is nothing compared to the amount of data needed to train a LM
Solution: A few data is required for SFT

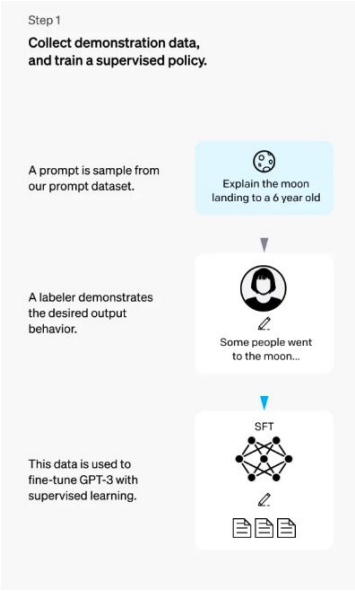


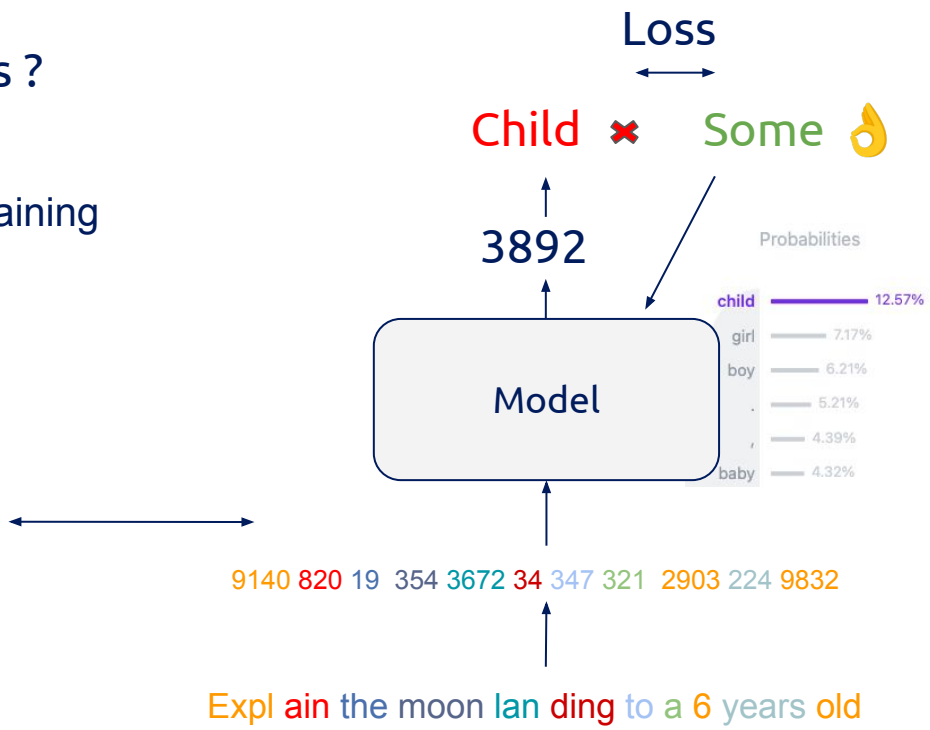
Zhou & Al 2023, LIMA: Less Is More for Alignment

© Florian Bastin, 2024. Tous droits réservés.

How Supervised Fine Tuning Works ?

Same process than the language model training





Open AI, 2022, Aligning language models to follow instructions [Blog]

© Florian Bastin, 2024. Tous droits réservés.

Reinforcement Learning from Human Feedback - RLHF

SFT Limitations:

- Behavior cloning
- Human abilities to answer perfectly to a given question
- Hallucination if answer from human not in training data
- Data collection cost

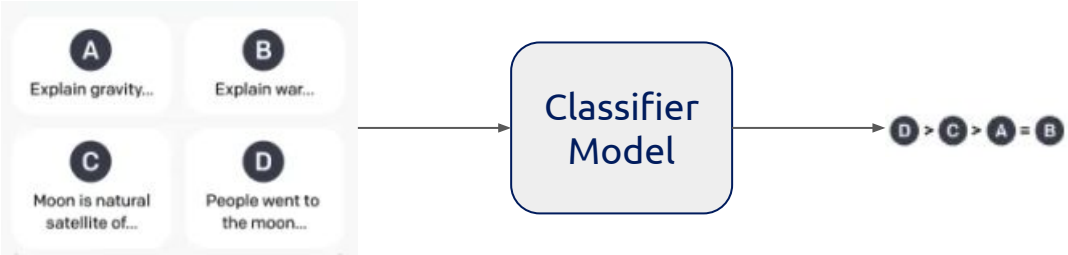
© Florian Bastin, 2024. Tous droits réservés.

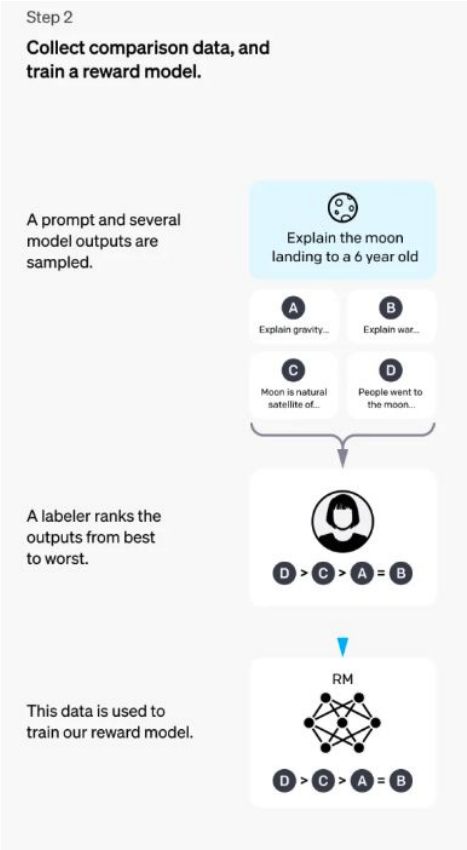
Reinforcement Learning from Human Feedback - RLHF

Idea:

1. From a question, generate multiples answers
2. Ask a human to classify answers
3. Train a reward model to learn these preferences

Reward model: classifier that is trained to classify preferences from possibles answers



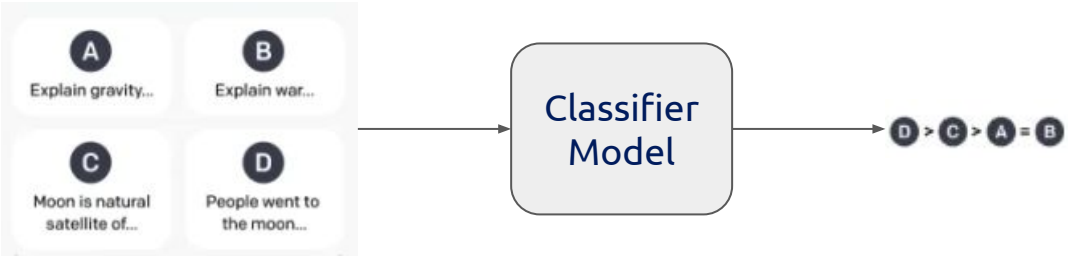


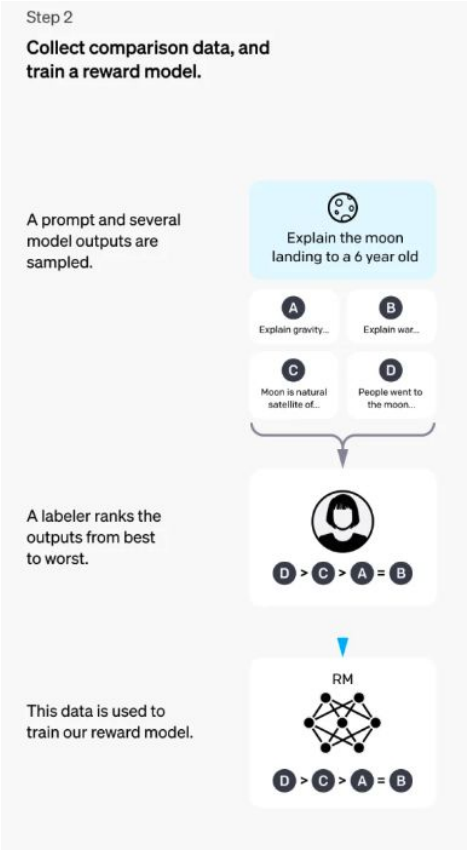
Open AI, 2022, Aligning language models to follow instructions [Blog]

© Florian Bastin, 2024. Tous droits réservés.

Reward model

$$\Pr(i > j) = \frac{p_i}{p_i + p_j} = \frac{\exp(R(x, \hat{y}_i))}{\exp(R(x, \hat{y}_i)) + \exp(R(x, \hat{y}_j))}$$

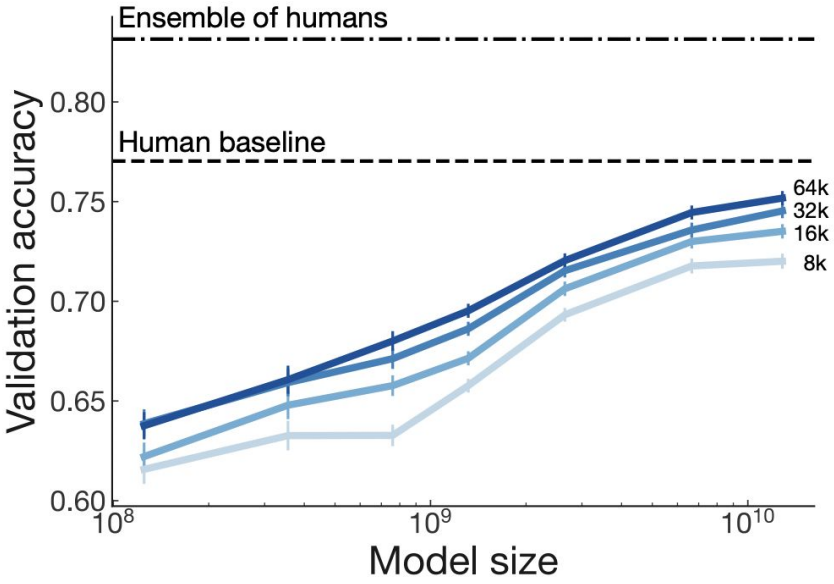




Open AI, 2022, Aligning language models to follow instructions [Blog]

© Florian Bastin, 2024. Tous droits réservés.

Reward model



Stiennon et Al, 2020, Learning to summarize from human feedback

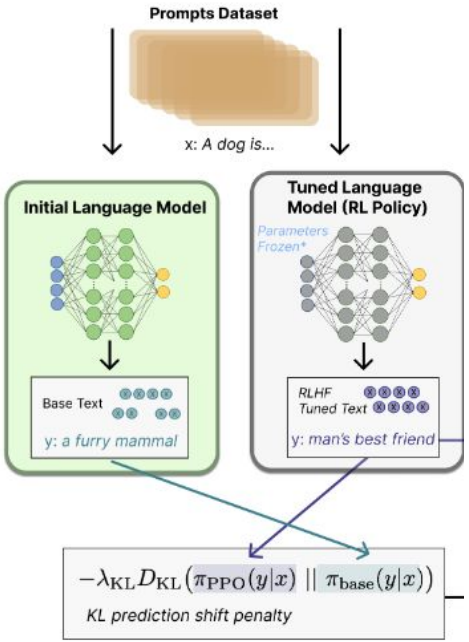
© Florian Bastin, 2024. Tous droits réservés.

Training RL model

- Also transformer based LM
- Variation in sizes used (relative to policy)
- Outputs scalar from input text

Prevent over optimization

$$\mathbb{E}_{\hat{y}\sim p_\theta(\hat{y}|x)}\left[R(x,\hat{y}) - \beta \log \frac{p_\theta(\hat{y}|x)}{p_{\text{ref}}(\hat{y}|x)}\right]$$



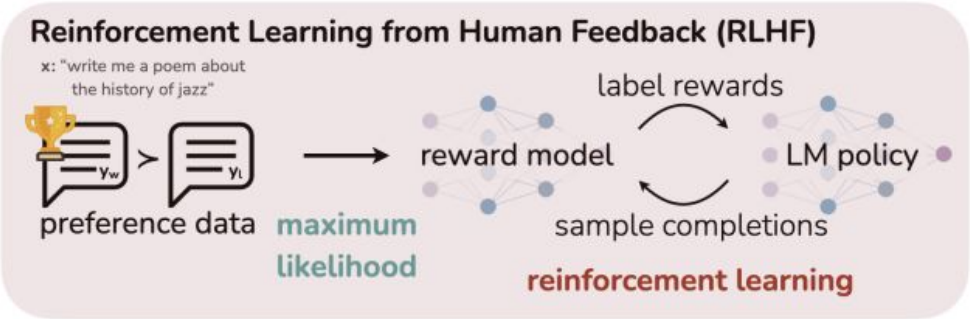
Lambert, 2022, Illustrating Reinforcement Learning from Human Feedback (RLHF) [Blog]

© Florian Bastin, 2024. Tous droits réservés.

DPO

- PPO is much more complex (clipping, rollouts, outer loops) than in theory
- Maximize the desired output, minimize the other

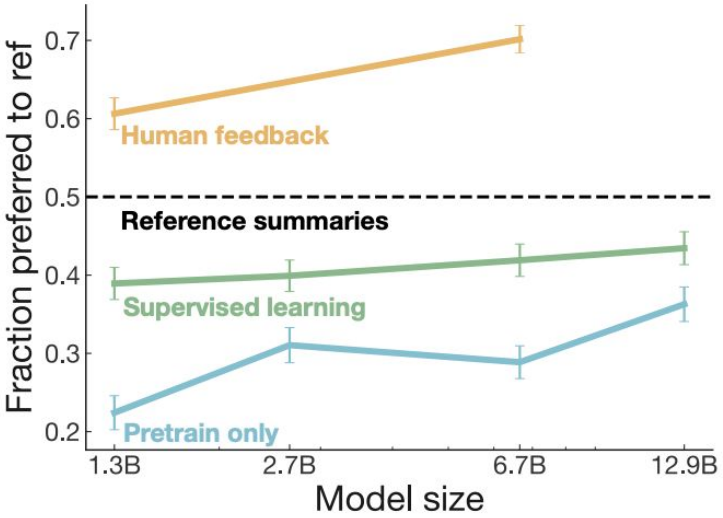
$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta\log\frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right]$$



Rafael Rafailov, 2024, Direct Preference Optimization:. Your Language Model is Secretly a Reward Model Paper

© Florian Bastin, 2024. Tous droits réservés.

RLHF gains



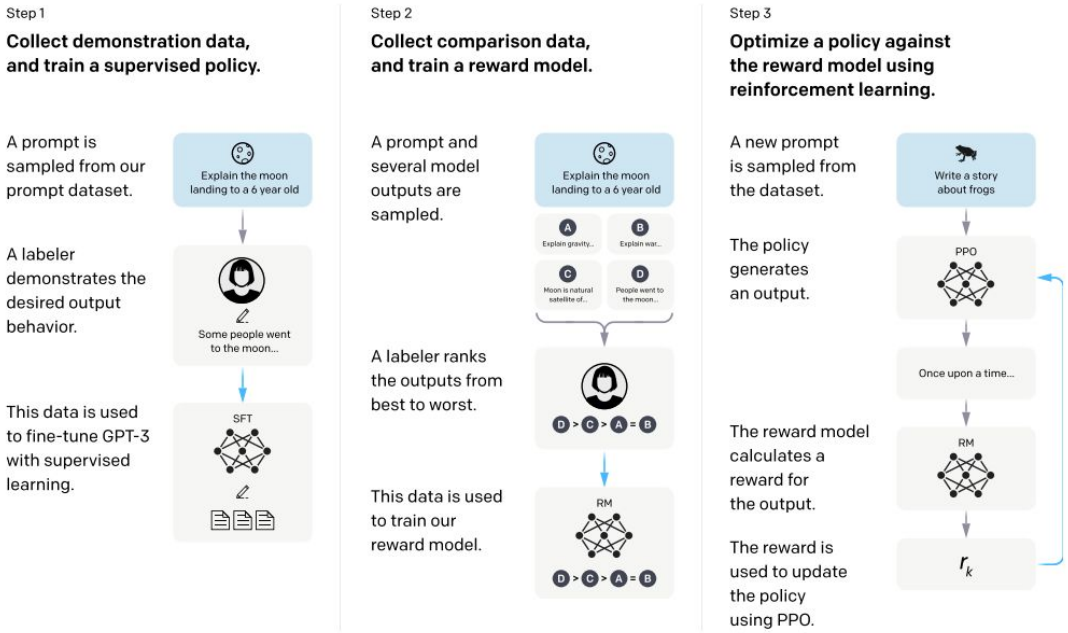
Method	Simulated Win-rate (%)	Human Win-rate (%)
GPT-4*†	79.0 ± 1.4	69.8 ± 1.6
ChatGPT*†	61.4 ± 1.7	52.9 ± 1.7
PPO	46.8 ± 1.8	55.1 ± 1.7
DPO	46.8 ± 1.7	-
Best-of-1024	45.0 ± 1.7	50.7 ± 1.8
Expert Iteration	41.9 ± 1.7	45.7 ± 1.7
SFT 52k	39.2 ± 1.7	40.7 ± 1.7
SFT 10k	36.7 ± 1.7	44.3 ± 1.7
Binary FeedME	36.6 ± 1.7	37.9 ± 1.7
Quark	35.6 ± 1.7	-
Binary Reward Conditioning	32.4 ± 1.6	-
Davinci001*	24.4 ± 1.5	32.5 ± 1.6
LLaMA 7B*	11.3 ± 1.1	6.5 ± 0.9

Nisan Stiennon & Al, 2020, Learning to summarize from human feedback

Dubois* & Al, 2024, Alpaca Farm: A Simulation Framework for Methods that Learn from Human Feedback

© Florian Bastin, 2024. Tous droits réservés.

Chat GPT

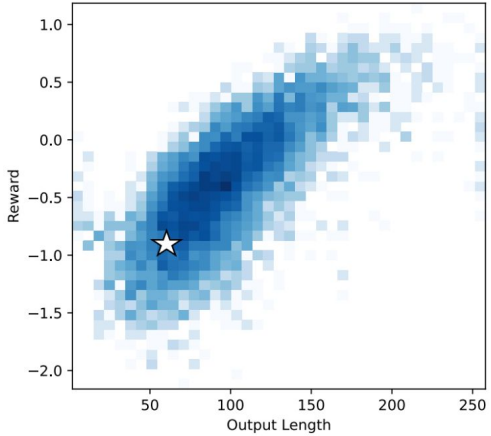


Open AI, Aligning human preferences

© Florian Bastin, 2024. Tous droits réservés.

RLHF challenges

- Answer preference is not trivial
- RLHF increases answer size
- Humans do not agree (agree with themselves only 66% of the time)
- Human have lot of variance, model have no variance
  - Ask LLM preferences instead of human preferences



Question: *Why don't adults roll off the bed?*

☆ **SFT (Before); 59 tokens**

Adults typically do not roll off of the bed because they have developed the muscle memory to keep their bodies from involuntarily moving during sleep and maintaining proper posture.

RLHF (After); 243 tokens: **Similar output, but much longer / more details**

Adults generally do not roll off of the bed because they have developed muscle control and awareness which helps them to remain upright while sleeping. Additionally, most adults find it uncomfortable or even painful to move around in their sleep, so rolling off the bed would be difficult without waking up first. There may also be safety concerns such as falling out of bed and potentially hurting oneself if one were to lose balance or fall from the ...

▸ **Annotation Guidelines**

Instruction

Tell me about self driving cars

example

Input

AI Response 1

Self driving cars are vehicles that are capable of detecting their surroundings and navigating without any input from a human driver. They use a variety of sensors, such as cameras, radar, and lidar, to detect objects in their environment and use advanced algorithms to make decisions about how to maneuver around them. Self driving cars are becoming increasingly popular and are being used in a variety of applications, such as ride-hailing services and autonomous delivery services.

AI Response 2

Self driving cars are cars that are equipped with sensors, cameras, and software, which allows them to navigate without the need for a driver. Self driving cars are able to monitor their environment and navigate without the assistance of a driver. Self driving cars use a combination of technologies such as GPS, radar, lidar, and computer vision to detect their environment and avoid obstacles. They are also able to update their maps and adjust their routes to account for variables such as traffic and construction.

The main benefit of self driving cars is their ability to reduce crashes and save lives. Autonomous vehicles have the potential to save up to 30,000 lives annually. They are also more energy efficient than traditional vehicles, as they do not require a driver who has to maintain a minimum speed. Autonomous vehicles also create new opportunities for increased mobility, allowing those who are unable to drive to get around.

Rating

- ○ Response 1 is better.
- ○ Response 1 is only slightly better. (only pick this if it's truly close)
- ○ Response 2 is only slightly better. (only pick this if it's truly close)
- ○ Response 2 is better.

Singhal & Al, 2024, A Long Way to Go: Investigating Length Correlations in RLHF

Stanford CS229 I Machine Learning I Building Large Language Models (LLMs) [Youtube]

© Florian Bastin, 2024. Tous droits réservés.

Evaluation

How to evaluate a model like Chat GPT ?

- Different methods (DPO, PPO, SFT) can't be compared
- Models are not calibrated
- A large diversity of evaluation to cover

Model	Size	Arena Elo	MMLU	Context Window	License
o1-preview	72B +	1355	92.3	128K	Proprietary
ChatGPT-4o-latest	72B +	1335	88.7	128K	Proprietary
Llama-3.1-405b-Instruct	72B +	1266	88.6	128K	Llama 3.1 Community
Llama-3.1-70b-Instruct	40B-72B	1248	86	128K	Llama 3.1 Community
Gemma-2-27b-it	24B-40B	1217	75.2	8K	Gemma license
Llama-3.1-8b-Instruct	4B-24B	1171	73	128K	Llama 3.1 Community
Phi-3-Mini-4k-Instruct	1B-4B	1071	70.9	4K	MIT

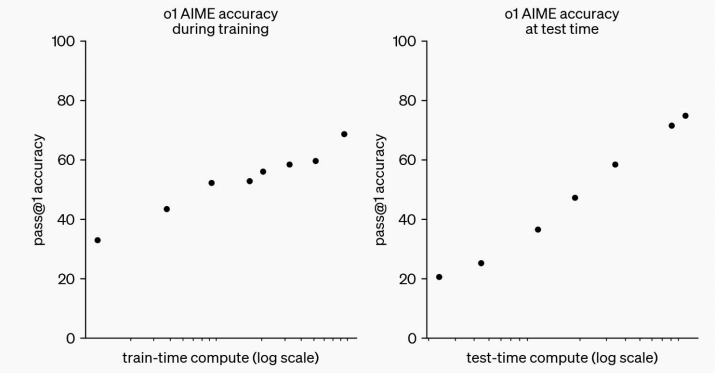
Chatbot Arena, Open LM

© Florian Bastin, 2024. Tous droits réservés.

OpenAI o1: “Streaming is dead, long live Chain of Thought”

- Chain of thought (COT)
- Increase test time compute

“Our large-scale reinforcement learning algorithm teaches the model how to think productively using its chain of thought in a highly data-efficient training process. We have found that the performance of o1 consistently improves with more reinforcement learning (train-time compute) and with more time spent thinking (test-time compute). The constraints on scaling this approach differ substantially from those of LLM pretraining, and we are continuing to investigate them.”



o1 performance smoothly improves with both train-time and test-time compute

Open AI, 2024, Learning to Reason with LLMs [Blog]

© Florian Bastin, 2024. Tous droits réservés.

OpenAI o1 vs GPT 4o

Prompt:
From the following sentence: Transformers are encoder decoder models
Apply the following steps:
- Create a manim code to display this sentence where each character has a different color
- Iterate through the sentence merging commons pairs as done n the Byte Pair Encoding system
- Change the colors of new pair
- Continue until all commons pair are made
- Update at each step the manim code
- Edit the previous code to not keep one color after merging on the merge pair. The selected color should be the one with the highest number of letters
- Edit the code at the final stage to change color if two adjacent different pair have same color

Open AI o1

GPT 4o

© Florian Bastin, 2024. Tous droits réservés.

Conclusion

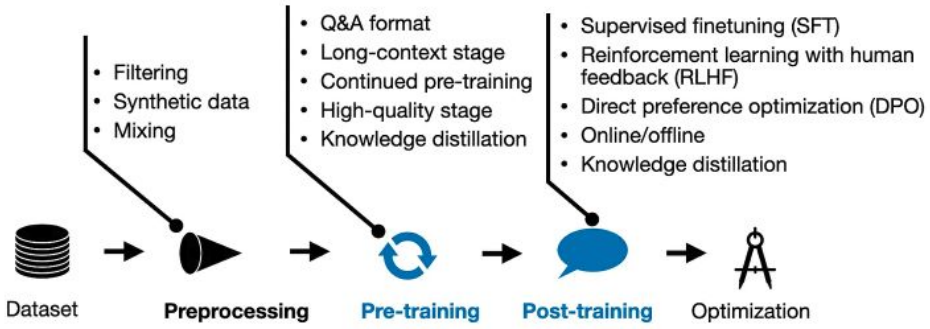
Building GPT 3

- Data preprocessing is a very important step to get quality data
- GPT 3 training consists of two phases (pre and post training)
- Supervised Fine Tuning is the first step of post training phase (ask a human to write the answer)
- RLHF helps the model to align with human preferences
- DPO is the new methods for Alignment, replacing RLHF

General knowledges

Data size and model size depends on compute resources (Scaling Laws, Chinchilla).
OpenAI o1 improves efficiency with longer RLHF training and answer inference time (COT)

"SFT+DPO approach seems to be the most popular preference tuning strategy at the moment due to the ease of use compared to other methods, such as RLHF with PPO."



An overview of the LLM development and training pipeline, with a focus on new pre-training and post-training methodologies discussed in this article

Sebastian Raschka, 2024, New LLM Pre-training and Post-training Paradigms [Blog]

© Florian Bastin, 2024. Tous droits réservés.

Conclusion

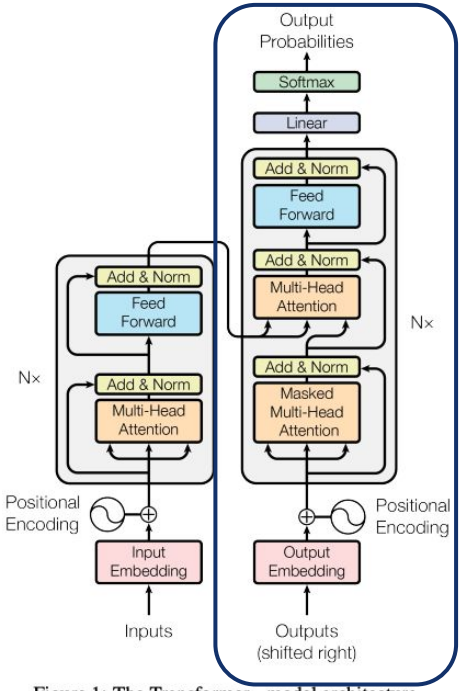
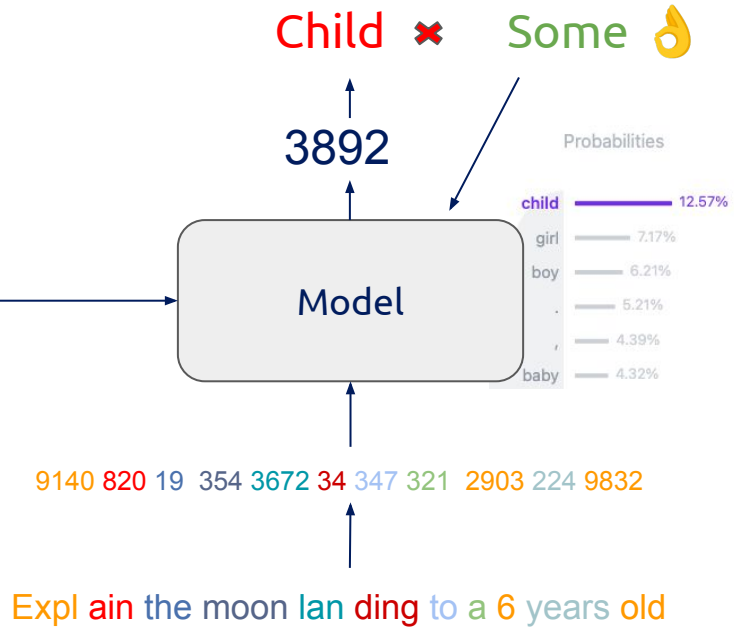


Figure 1: The Transformer - model architecture.



© Florian Bastin, 2024. Tous droits réservés.

Quiz Kahoot

Go to : https://kahoot.com/
Click Play
Enter code: XXX XXX
Enter your name

© Florian Bastin, 2024. Tous droits réservés.